

EXPERIMENT 4

Write a Python Program for the Crypt-Arithmetic Problem

Aim

To develop a Python program to solve a **Crypt-Arithmetic Problem** using **Backtracking Search**, where each letter represents a unique digit and the arithmetic equation is satisfied.

Objective

- To understand Constraint Satisfaction Problems (CSP) in Artificial Intelligence.
- To apply the Backtracking algorithm to solve Crypt-Arithmetic problems.
- To assign unique digits to letters while satisfying arithmetic constraints.
- To ensure that no two letters have the same digit and leading letters are not assigned zero.

Theory

A **Crypt-Arithmetic Problem** is a mathematical puzzle in which digits are replaced by letters. Each letter represents a unique digit from **0 to 9**, and the objective is to find the correct digit assignment that satisfies the given arithmetic equation.

A famous example is:

```
SEND
+ MORE
-----
MONEY
```

Where:

- Each letter represents a unique digit.
- No two letters can have the same digit.
- The first letter of a number cannot be zero.

Since the number of possible combinations is very large, **Backtracking Search** is used to systematically assign digits and eliminate invalid assignments.

Algorithm

Step 1: Define the crypt-arithmetic equation.

Step 2: Identify all unique letters.

Step 3: Generate possible digit assignments (0–9).

Step 4: Assign digits to letters one by one.

Step 5: Check whether:

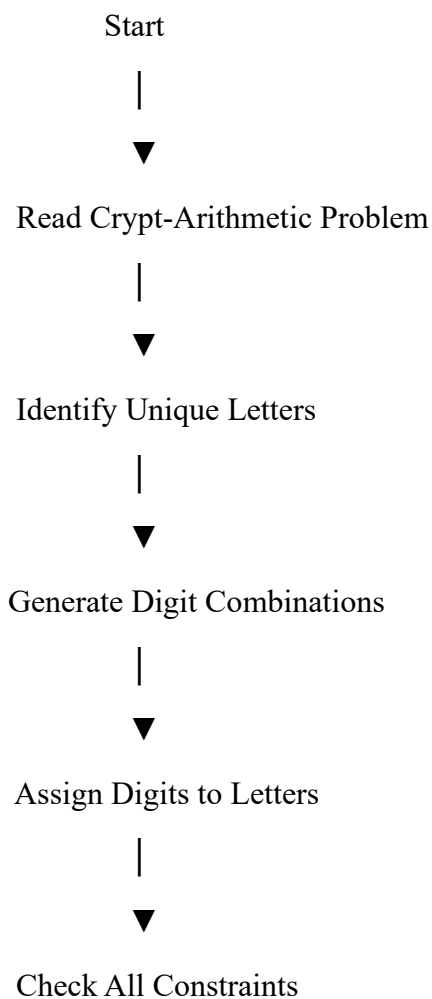
- Each letter has a unique digit.
- Leading letters are not assigned zero.
- The arithmetic equation is satisfied.

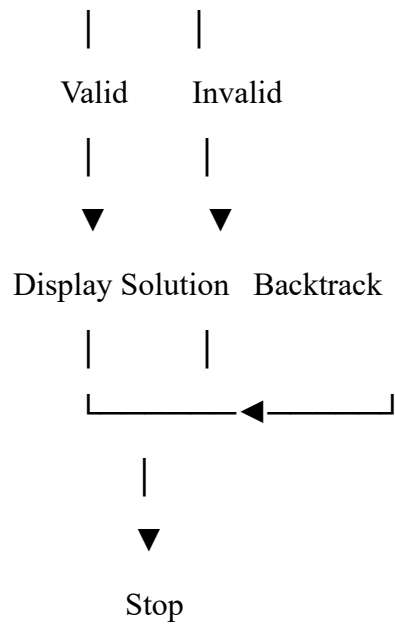
Step 6: If the assignment is valid, display the solution.

Step 7: Otherwise, backtrack and try another digit assignment.

Step 8: Continue until a valid solution is found.

Flowchart





Python Program

```
from itertools import permutations
```

```
letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
```

```
for p in permutations(range(10), len(letters)):
```

```
    S, E, N, D, M, O, R, Y = p
```

```
    if S == 0 or M == 0:
```

```
        continue
```

```
    SEND = 1000*S + 100*E + 10*N + D
```

```
    MORE = 1000*M + 100*O + 10*R + E
```

```
    MONEY = 10000*M + 1000*O + 100*N + 10*E + Y
```

```
    if SEND + MORE == MONEY:
```

```
print("Solution Found\n")
```

```
print("SEND =", SEND)
```

```
print("MORE =", MORE)
```

```
print("MONEY =", MONEY)
```

```
print("\nLetter Assignments")
```

```
print("S =", S)
```

```
print("E =", E)
```

```
print("N =", N)
```

```
print("D =", D)
```

```
print("M =", M)
```

```
print("O =", O)
```

```
print("R =", R)
```

```
print("Y =", Y)
```

```
break
```

Sample Output

Solution Found

SEND = 9567

MORE = 1085

MONEY = 10652

Letter Assignments

S = 9

E = 5

$$N = 6$$

$$D = 7$$

$$M = 1$$

$$O = 0$$

$$R = 8$$

$$Y = 2$$

Explanation of Output

The program assigns unique digits to each letter:

Letter Digit

S 9

E 5

N 6

D 7

M 1

O 0

R 8

Y 2

Verification:

$$\begin{array}{r} 9567 \\ + 1085 \\ \hline 10652 \end{array}$$

Since the equation is satisfied, the solution is correct.

Advantages

- Finds a valid solution systematically.
- Eliminates invalid assignments through backtracking.

- Ensures all constraints are satisfied.
- Suitable for solving Constraint Satisfaction Problems.

Applications

- Artificial Intelligence.
- Constraint Satisfaction Problems (CSP).
- Puzzle solving.
- Scheduling and planning.
- Logic-based reasoning systems.

Result

The Python program successfully solved the **Crypt-Arithmetic Problem** using the **Backtracking Algorithm** by assigning unique digits to letters while satisfying all arithmetic and uniqueness constraints.

Conclusion

The **Crypt-Arithmetic Problem** is a classic **Constraint Satisfaction Problem (CSP)** in Artificial Intelligence. By applying the **Backtracking Algorithm**, the program efficiently searches through possible digit assignments and eliminates invalid combinations until a valid solution is found. This experiment demonstrates the effectiveness of backtracking in solving complex logical and combinatorial problems while satisfying multiple constraints simultaneously.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Varshini M\AppData\Local\Programs\Microsoft VS Code> & "C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs\EXP1.py"
```

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

Letter Assignments

```
S = 9
E = 5
N = 6
D = 7
M = 1
O = 0
R = 8
Y = 2
```

C:\Users\Varshini M\OneDrive\Desktop\Here!!!\ARTIFICIAL INTELLIGENCE\VSC EXPs\EXP1.py

```
1  from itertools import permutations
2
3  letters = ('S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y')
4
5  for p in permutations(range(10), len(letters)):
6
7      S, E, N, D, M, O, R, Y = p
8
9      if S == 0 or M == 0:
10         continue
11
12     SEND = 1000*S + 100*E + 10*N + D
13     MORE = 100*M + 10*O + 10*R + E
14     MONEY = 1000*M + 100*O + 10*N + 10*E + Y
15
16     if SEND + MORE == MONEY:
17
18         print("Solution Found\n")
19
20         print("SEND =", SEND)
21         print("MORE =", MORE)
22         print("MONEY =", MONEY)
23
24         print("\nLetter Assignments")
25         print("S =", S)
26         print("E =", E)
27         print("N =", N)
28         print("D =", D)
29         print("M =", M)
30         print("O =", O)
31         print("R =", R)
32         print("Y =", Y)
33
34         break
35
```